TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

Modélisation UML

Chapitre 9: Le Diagramme de Déploiement

Auteur :

Mme Ines AGREBI



Plan

1- Qu’est ce qu’un diagramme de déploiement?

2- Mots clés

- Les nœuds
- Les composants
- Les connexions
- Les artefacts

3- Exercices d’application

Qu’est ce qu’un diagramme de déploiement

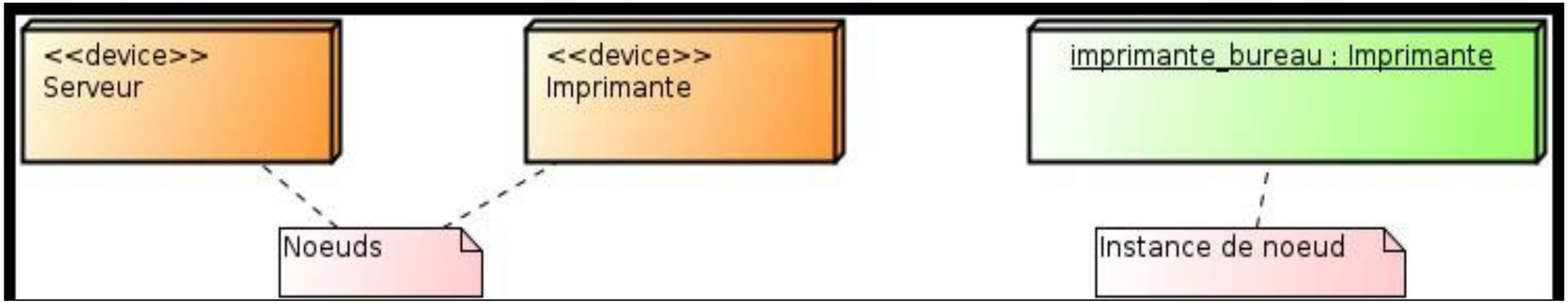
- En UML, un **diagramme de déploiement** est une vue statique qui sert à représenter l'utilisation de l'infrastructure physique par le système et la manière dont les **composants** du système sont répartis ainsi que leurs relations entre eux.
- Décrit la disposition physique des ressources matérielles qui composent le système et montre la répartition des composants sur ces matériels.
- Chaque ressource étant matérialisée par un **nœud**,
- Le diagramme de déploiement précise comment les composants sont répartis sur les nœuds et quelles sont les connexions entre les composants ou les nœuds.

Mots Clés

❖Les nœuds:

- Un nœud est une ressource matérielle du système. En général, cette ressource possède au minimum de la mémoire et parfois aussi des capacités de calcul (des ressources humaines ou des périphériques sont des ressources modélisées par des nœuds). Les ressources matérielles sont quelquefois représentées avec le stéréotype <<device>> (généralement plutôt les périphériques mais également tout système informatique comme par exemple les ordinateurs de bureau).

Exemple1:



Un nœud possède des attributs (quantité de mémoire, vitesse de processeur, marque, type…) que nous pouvons spécifier a l'intérieur du parallélépipède.

Exemple2:



❖Les composants: (élément logiciel)

- **Un composant** est une unité autonome représentée par un classeur structuré, stéréotypé *«component»*, comportant une ou plusieurs interfaces requises ou offertes. représentés par des boites rectangulaires avec deux rectangles sortant du côté gauche, sont les différentes parties du système étudié.
- Pour montrer qu'un composant est affecté sur un nœud, il faut :

- Soit en plaçant le composant dans le nœud.

- Soit en le reliant a l'aide d'une relation de dépendance (flèche en pointillées)

- stéréotypée ***«support»*** orientée du composant vers le nœud .

Exemple:



❖Les Connexions:

Sont principalement de deux types:

- **Associations:** représentées par de simples lignes sont des liens de communication, s'établissent entre les différents **composants** du système.
- **Dépendances:** représentées par des flèches en traits pointillés pointant vers le nœud en question.

Les chemins de communications:

- Les nœuds dans un diagramme de déploiement sont connectés entre eux par des lignes qui symbolisent un support de communication.

- Le chemin de communication est donc un lien qui permet de modéliser de façon simpliste la communication entre 2 nœuds (liaison Ethernet, USB, série…).

- Il est possible de faire figurer sur ce lien :

- Les cardinalités;
- Des contraintes entre accolades (pour indiquer par exemple qu'un accès est sécurisé) ;
- Le type de réseau et/ou son débit en l'indiquant comme un stéréotype…

Exemple:



❖Les Artefacts:

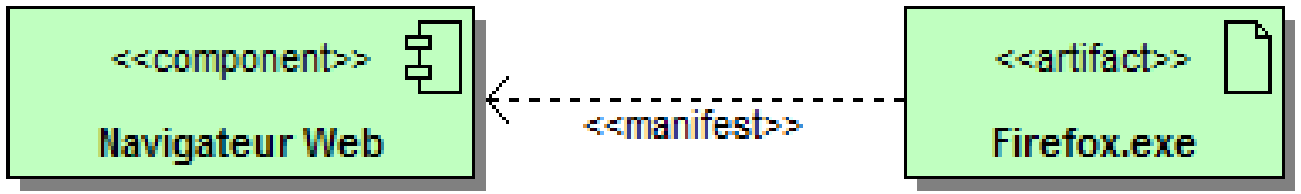
- Un **artefact** est une manière de définir un fichier, un programme, une bibliothèque ou une base de données construite ou modifiée dans un projet. Ces **artefacts** mettent en œuvre des collections de composants qui sont consommées ou créées durant l'une des étapes du processus de déploiement.

- En d'autres termes, L'artefact est le terme générique qui désigne n'importe quel élément produit du travail, c'est un élément concret et existant dans le monde réel (document, exécutable, fichier, base de donnée…). L'implémentation des modèles se fait sous forme d'artefacts. On dit que l'artefact est la manifestation du modèle qu'il implémente.

- Un artefact se représente par un rectangle contenant son nom et le stéréotype ***<<artifact>>*** (nous pouvons aussi y faire figurer le symbole).

- Un artefact qui est la manifestation d'un composant est relie a celui-ci par une relation de dépendance (flèche en pointillées) stéréotype ***<<manifest>>*** orienté de l'artefact vers le composant.

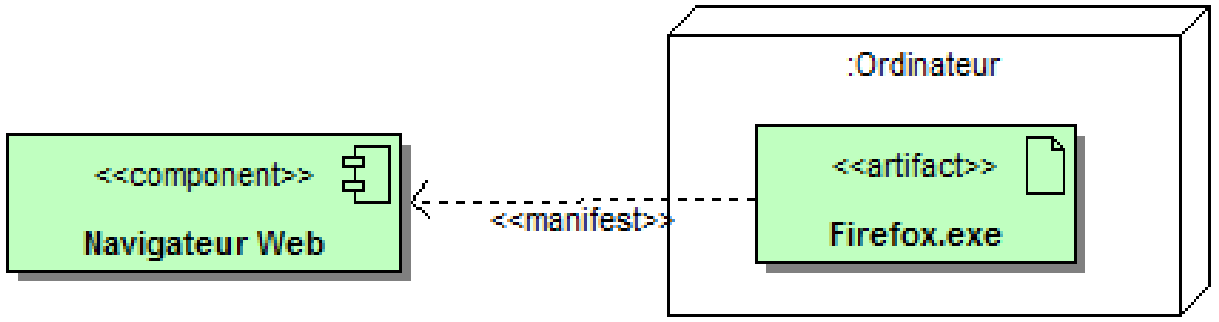
Exemple:



- L’artefact est placé soit :

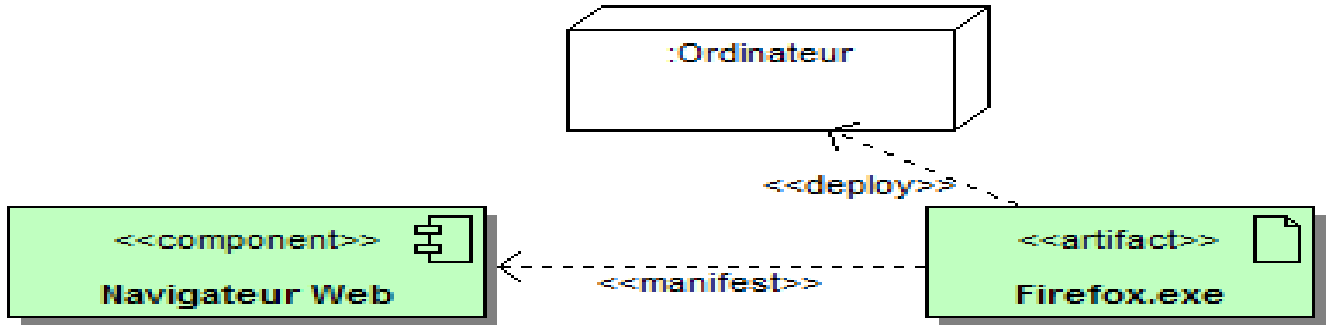
- A l’intérieur du nœud dans lequel il est déployé.

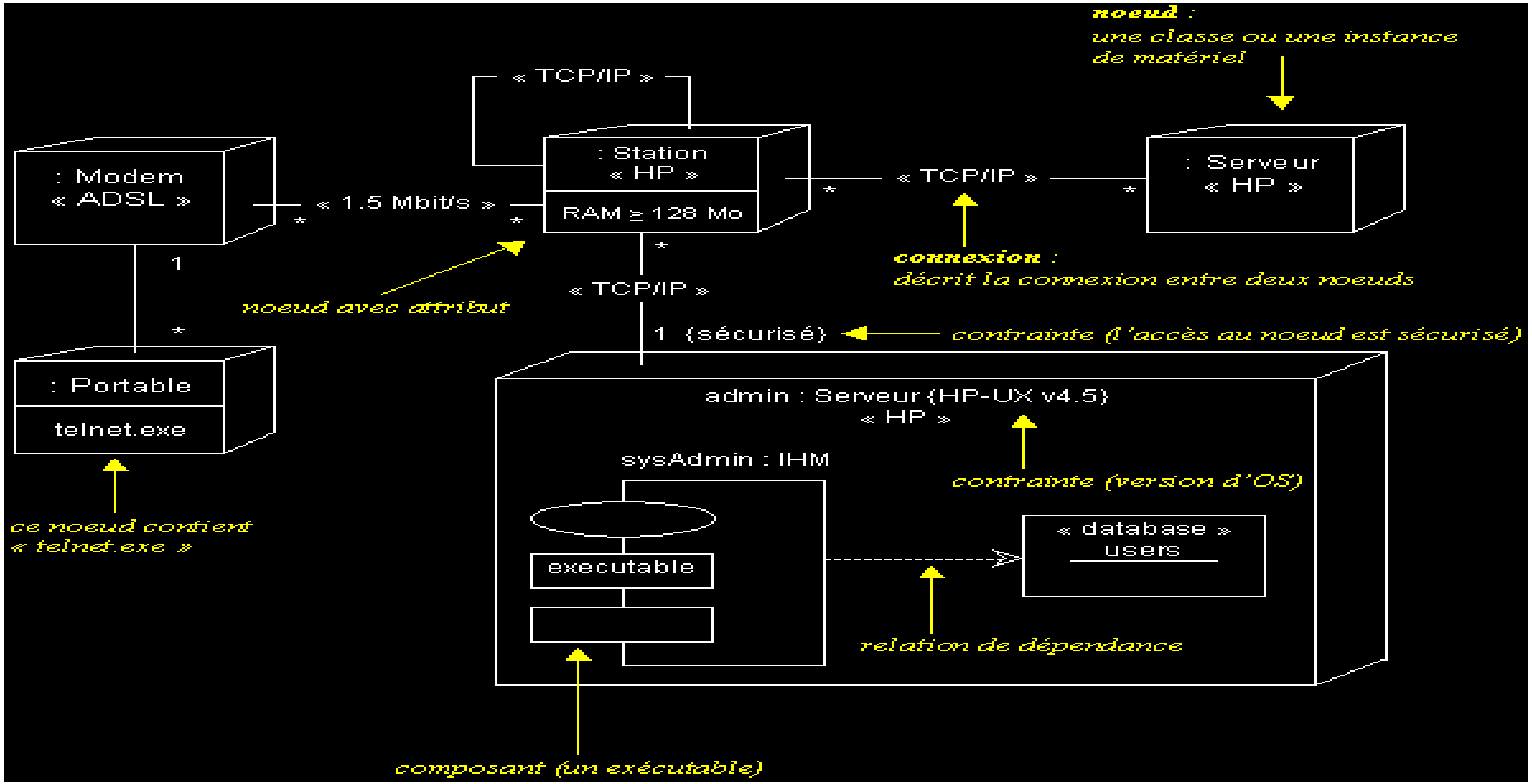
Formalisme et Exemple:



- A l’extérieur du nœud dans lequel il est déployé, mais relie a celui-ci par une relation de dépendance stéréotype ***<<deploy>>***.

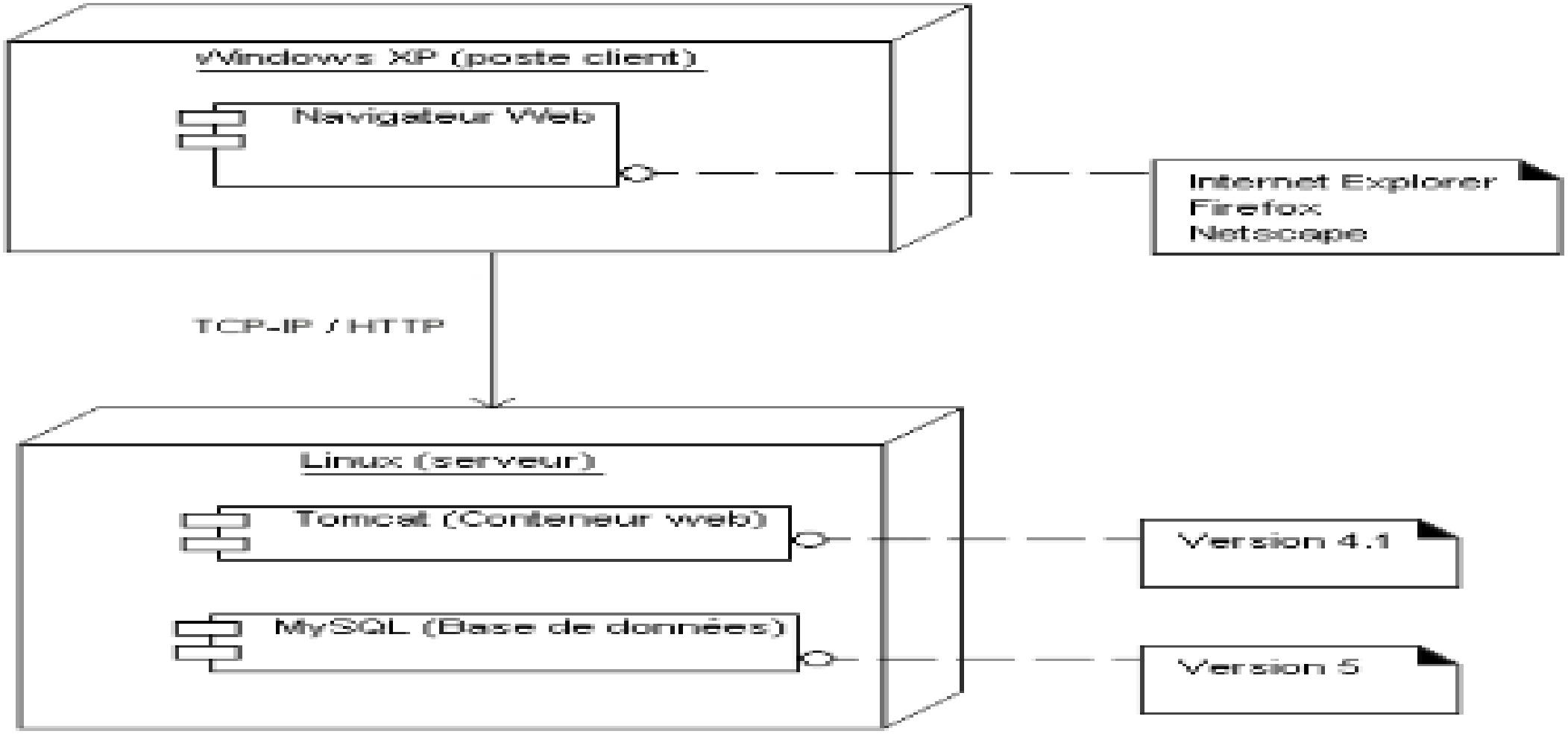
Formalisme et Exemple:




noeud :
une classe ou une instance de matériel
« TCP/IP »
: Station « HP »
RAM ≥ 128 Mo
: Modem « ADSL »
« 1.5 Mbit/s »
« TCP/IP »
: Serveur « HP »
connexion :
décrit la connexion entre deux noeuds
noeud avec attribut
« TCP/IP »
1 {sécurisé}
contrainte (l'accès au noeud est sécurisé)
admin : Serveur {HP-UX v4.5}
« HP »
sysAdmin : IHM
contrainte (version d'OS)
: Portable
telnet.exe
ce noeud contient « telnet.exe »
executable
« database »
users
relation de dépendance
composant (un exécutable)

Exercices d'application

❖ **Exercice 1:** Soit le diagramme UML suivant :



- C’est un diagramme de déploiement d’une application web.
- Ce diagramme est composé de deux parties physiques déployées sur deux nœuds :

➢La première partie constitue la partie cliente, soit une machine dont le système d’exploitation Windows XP. Un composant est affecté à ce nœud, qui est le navigateur web.

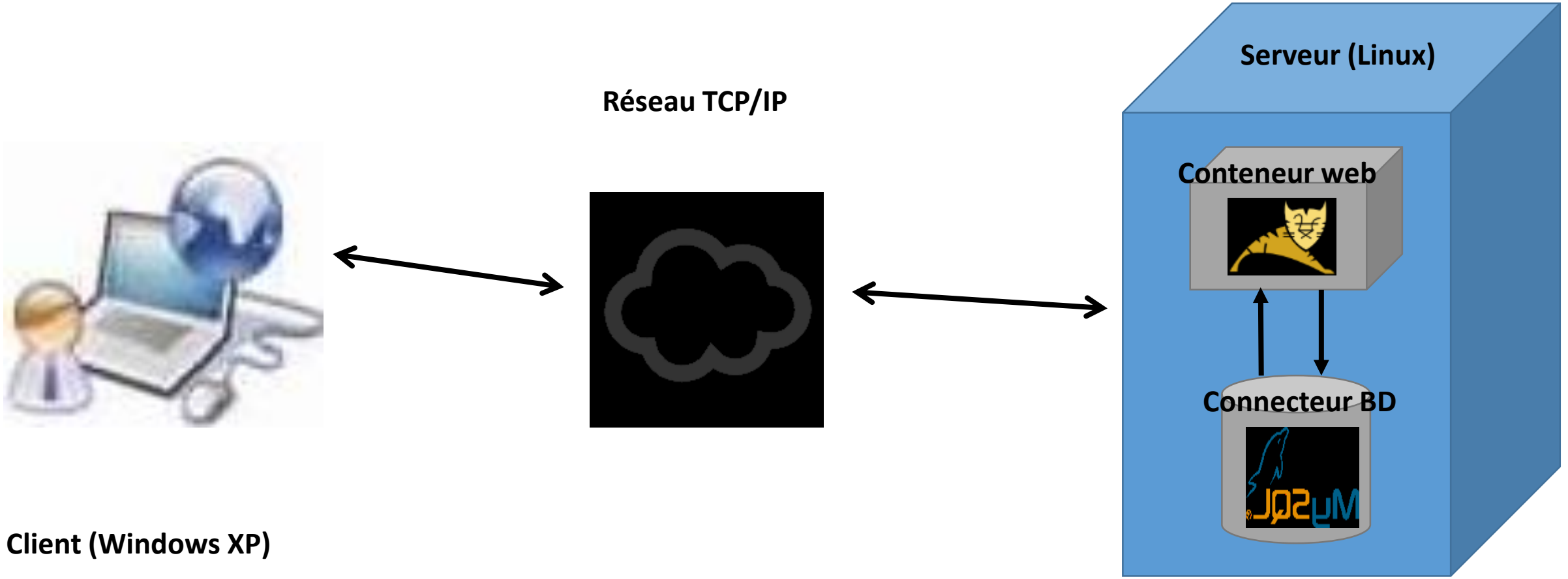
➢La deuxième partie est le serveur, installé sur une machine d’OS Linux.

✓Le premier composant associé est le conteneur Web, de type Tomcat, version 4.1.

✓L’autre composant fut le connecteur base de données, MySQL, version 5.

➡ Ces deux parties sont relié via une dépendance (car les composants sont déployés sur les nœuds directement) d’Internet selon le modèle TCP/IP.

- C’est une architecture **Client/ Serveur** (2-tiers). Soit le schéma suivant:



❖Exercice 2:

- La Société Nationale des Chemins de Fer Tunisiens (SNCFT) souhaite concevoir et développer une application de vente des billets et des abonnements. Cette application permet:

- à ses clients d'acheter des billets ou des abonnements

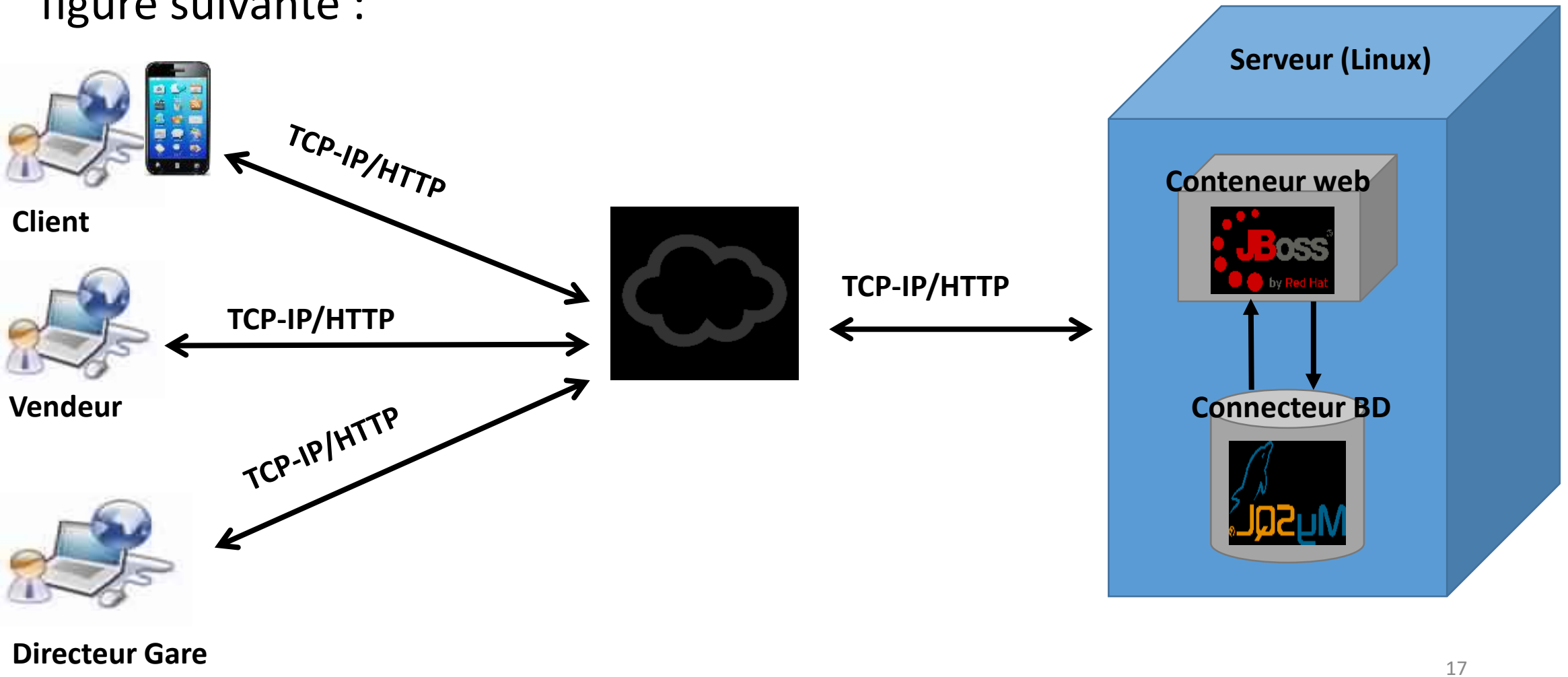
- à ses vendeurs aux guichets d'effectuer des ventes groupées des billets ou des abonnements.

- aux directeurs des gares de consulter les états des billets et d'élaborer des tableaux de bord.

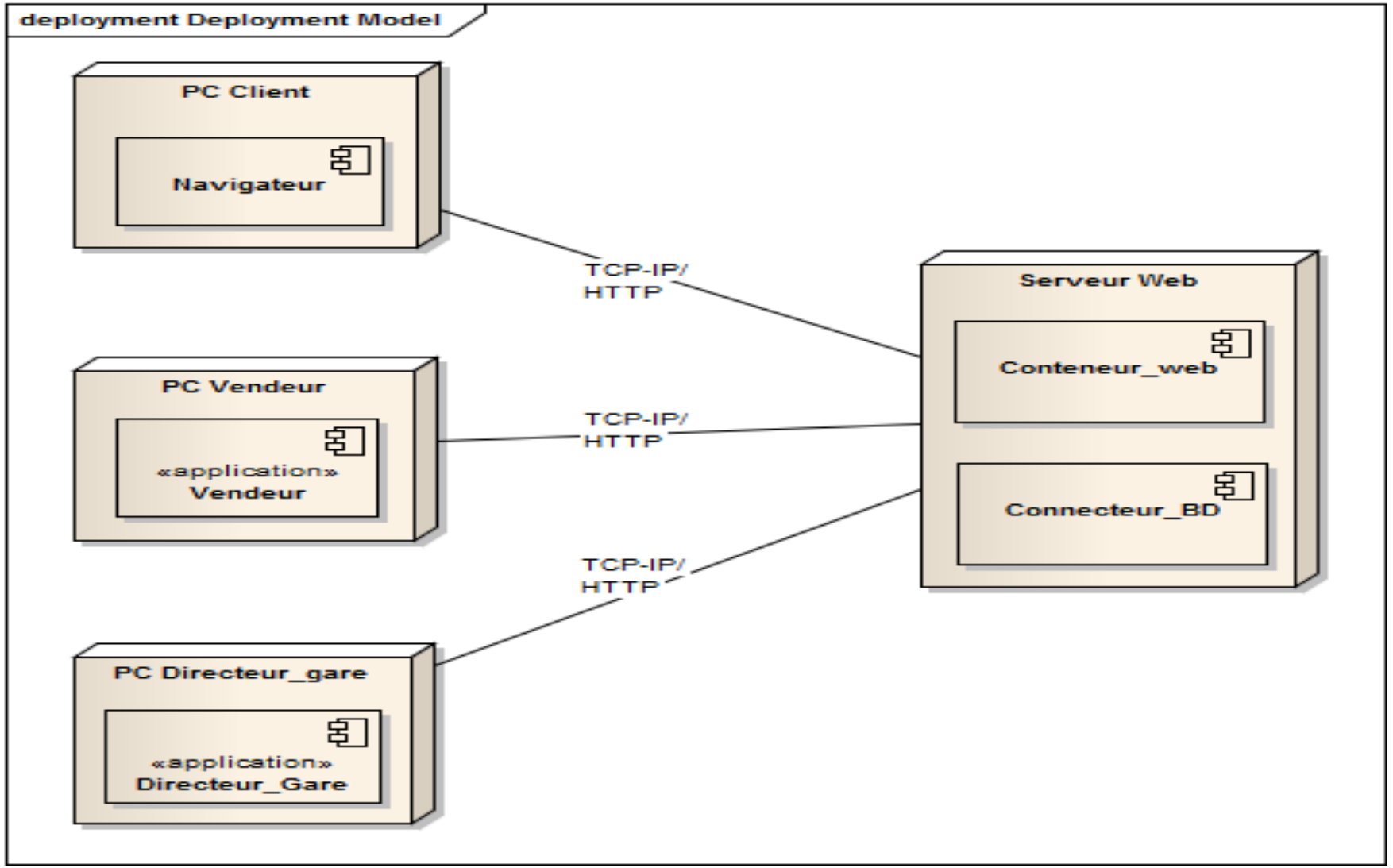
- Le paiement des billets et des abonnements s'effectue par carte bancaire.
- L'application doit offrir des interfaces personnalisées aux clients, aux vendeurs de guichets et aux directeurs des gares.

Travail à Faire: Proposer une architecture pour cette application et élaborer son diagramme de déploiement.

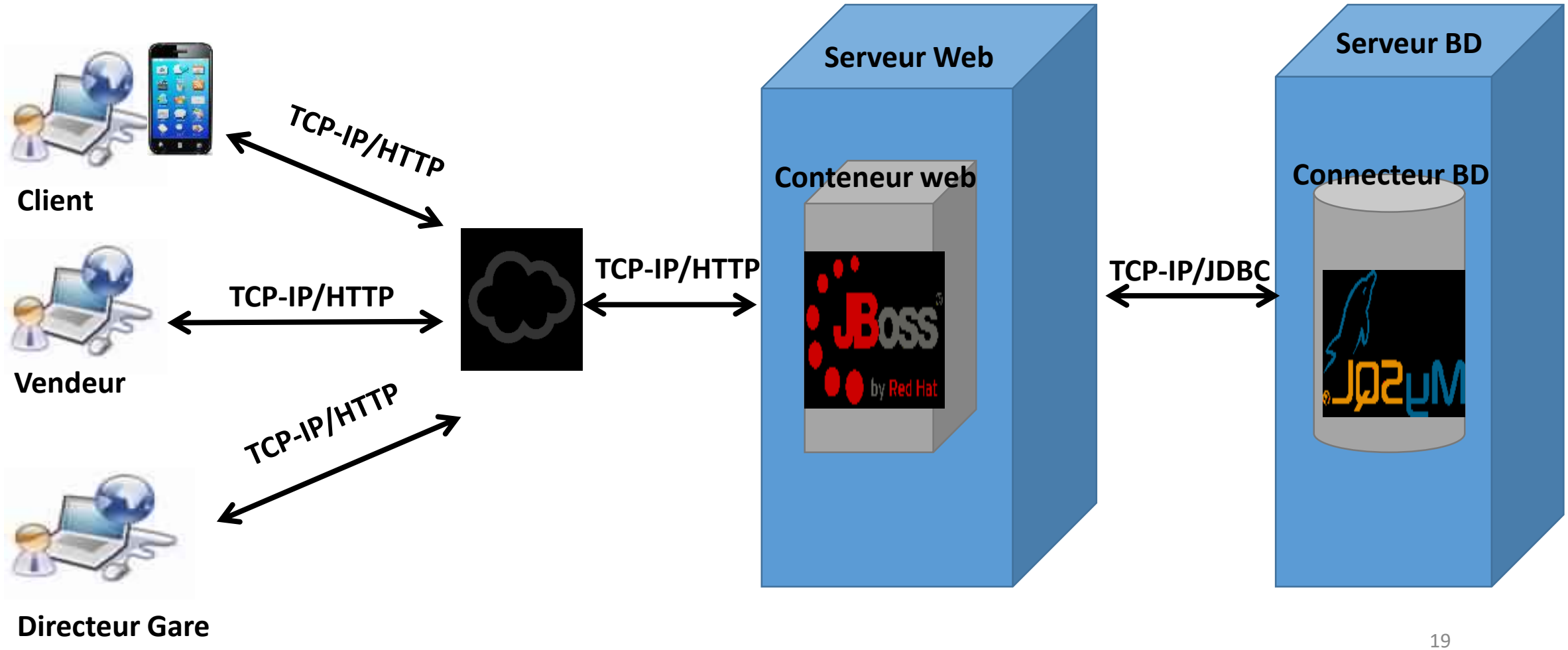
- Pour cette question, nous présentons deux solutions différentes :
- La première solution est une architecture 2-tiers présenté dans la figure suivante :



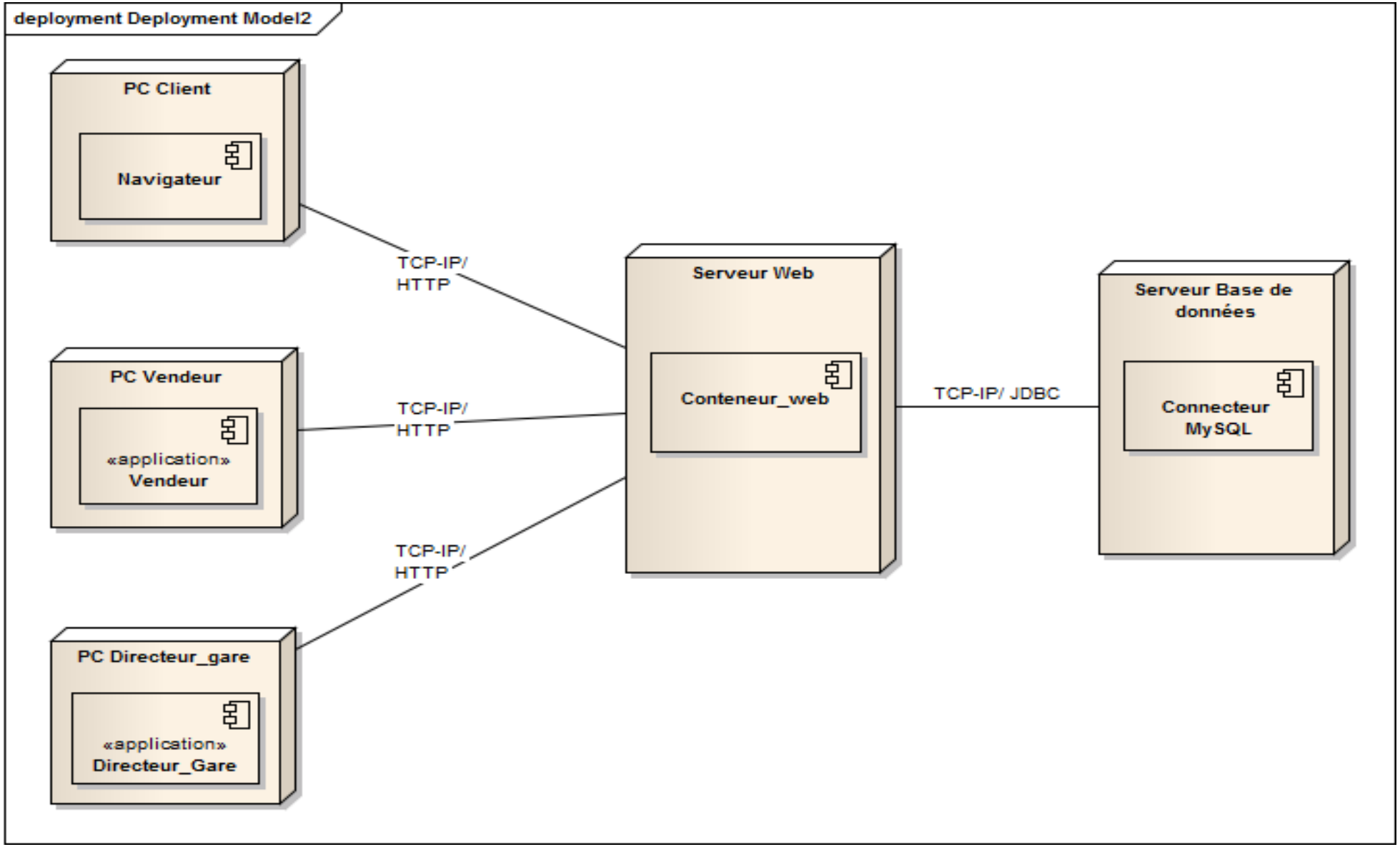
- Diagramme de déploiement correspondant à la première solution:



- La deuxième solution est une architecture 3-tiers qui permet d'éclater la partie serveur précédente sur deux tiers différents. Soit alors la figure suivante :



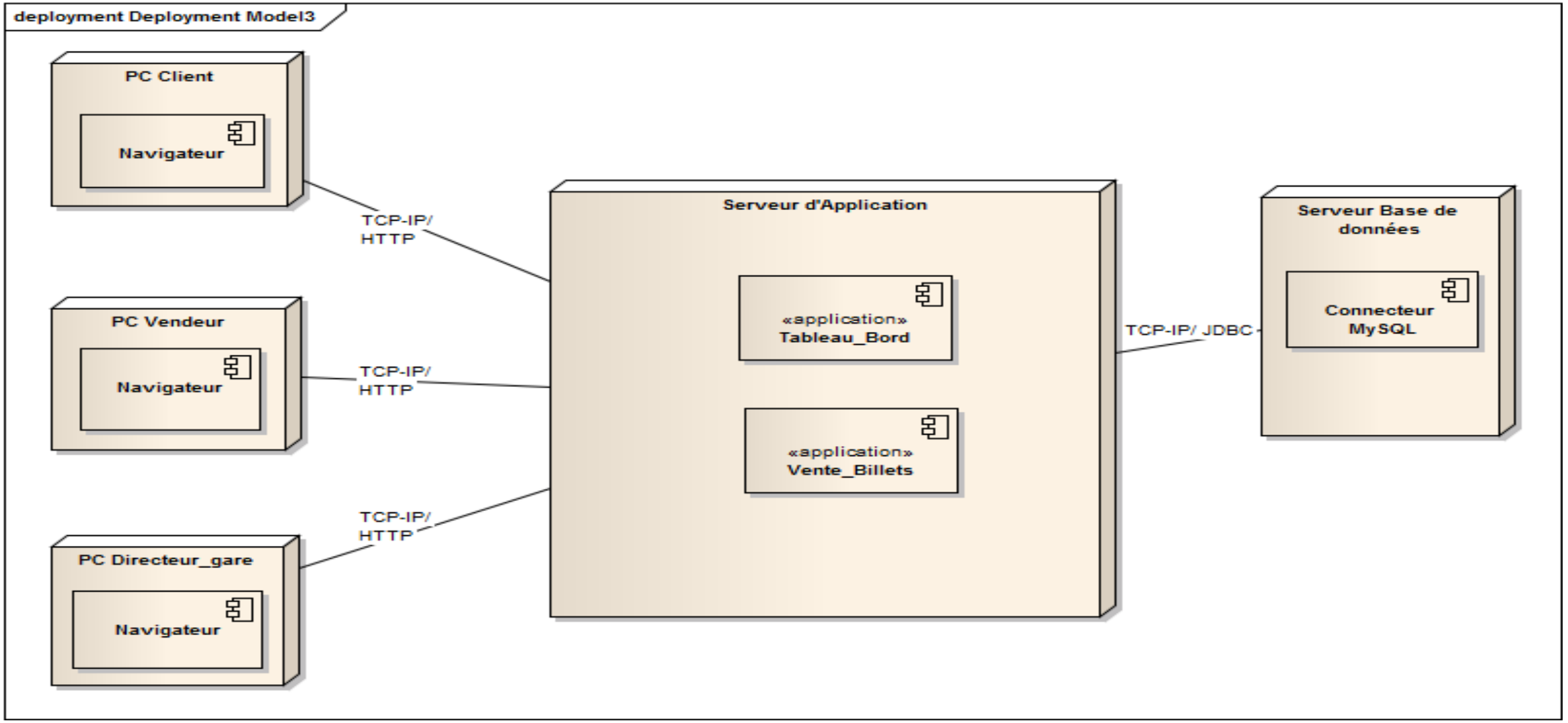
- Le diagramme de déploiement associé est le suivant :



- Soit une 3ème solution qui est 3-tiers aussi. Mais, en respectant un peu le cas du modèle MVC (en séparant juste la partie View sur les PCs du client, vendeur et directeur de la gare et en supposant que la partie Model et Controller est déployée sur le serveur d'application qui contient lui-même un conteneur web : cas du Jboss par exemple. Enfin le serveur bases de données contiendra juste la partie DAO : Data Access Object).
- Aussi, dans cette solution l'application la vente et celle de tableau du bord (on la suppose comme application prête, et juste le directeur de la gare va choisir les données nécessaires pour afficher le tableau avec graphiques et indicateurs de performances qui conviennent).

Remarque: DAO c'est les classes qui ont une relation directe avec les tables de base de données.

- Le diagramme de déploiement correspondant à la 3ième solution est le suivant:



❖Exercice 3:

- La société américaine YAPS vend des animaux de compagnie. Elle est implantée depuis plusieurs décennies dans le sud de la Californie, où ses principaux clients sont domiciliés.

Récemment, elle a ouvert son marché à d'autres états américains, ainsi qu'à l'étranger. Elle continue à exercer sa profession telle qu'elle le faisait à ses débuts. C'est-à-dire, qu'elle répertorie ses clients sur des fiches papiers indexées par le nom de famille, reçoit les commandes par fax et les chèques par courrier. Une fois le montant du chèque encaissé, YAPS envoie les animaux via la société de transport PetEx. Annuellement, YAPS envoie son catalogue d'animaux domestiques à ses clients. Elle trouve ses nouveaux clients au travers de publicités qu'elle envoie aussi par courrier.

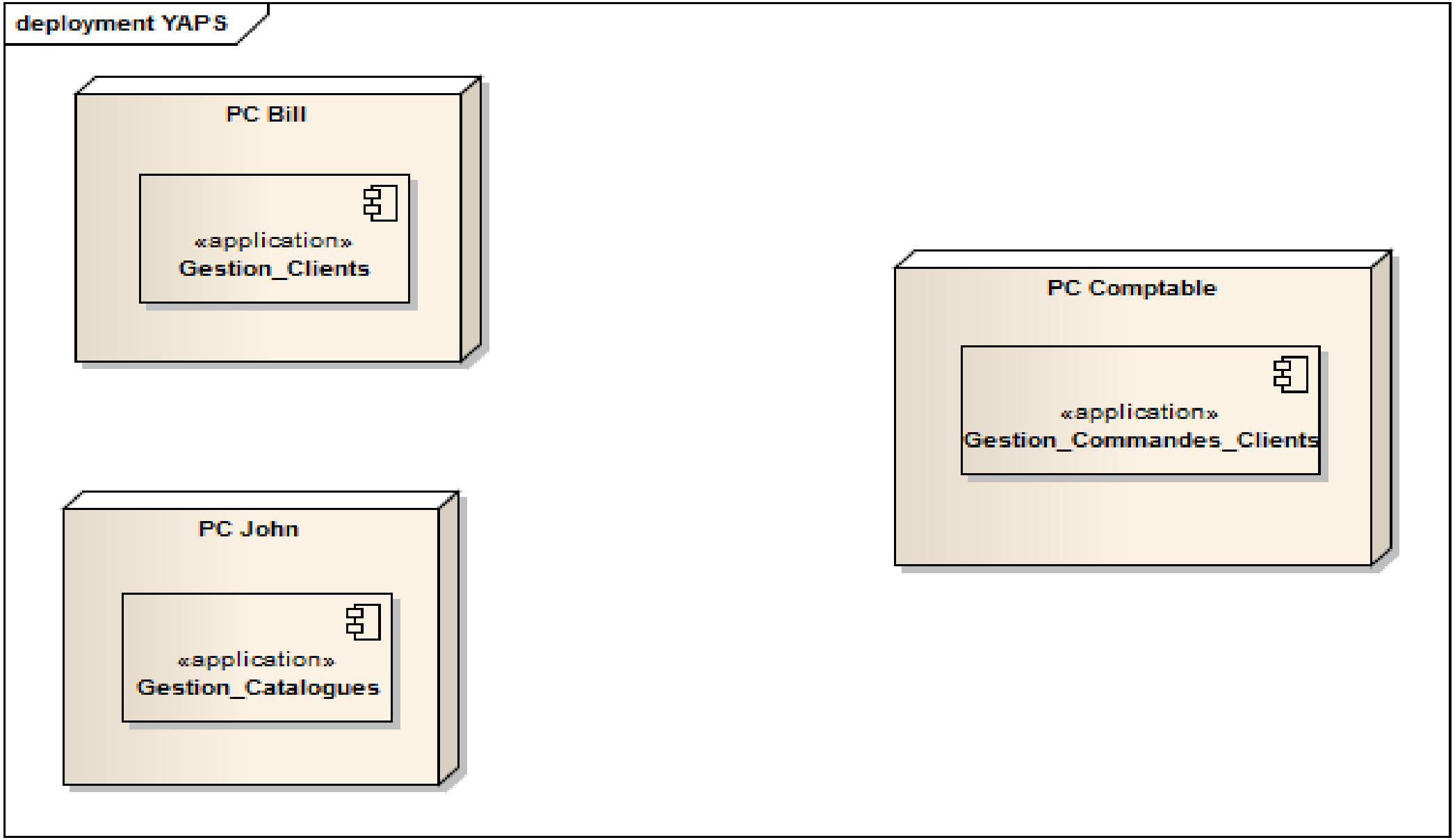
➢ *Configuration 1:*

- YAPS veut informatiser la gestion de ses clients et la gestion des catalogues. En effet, les clients et les catalogues sont de plus en plus nombreux. Elle voudrait saisir leurs coordonnées et pouvoir les modifier. Cette informatisation lui permettrait surtout de pouvoir retrouver les informations de ses clients et des catalogues plus rapidement. YAPS possède des PC avec Windows comme système d'exploitation.

✓La tâche de gestion des clients sera menée par Bill qui assure la relation clientèle.

✓La tâche de gestion des catalogues sera menée par John qui assure la gestion des catalogues.

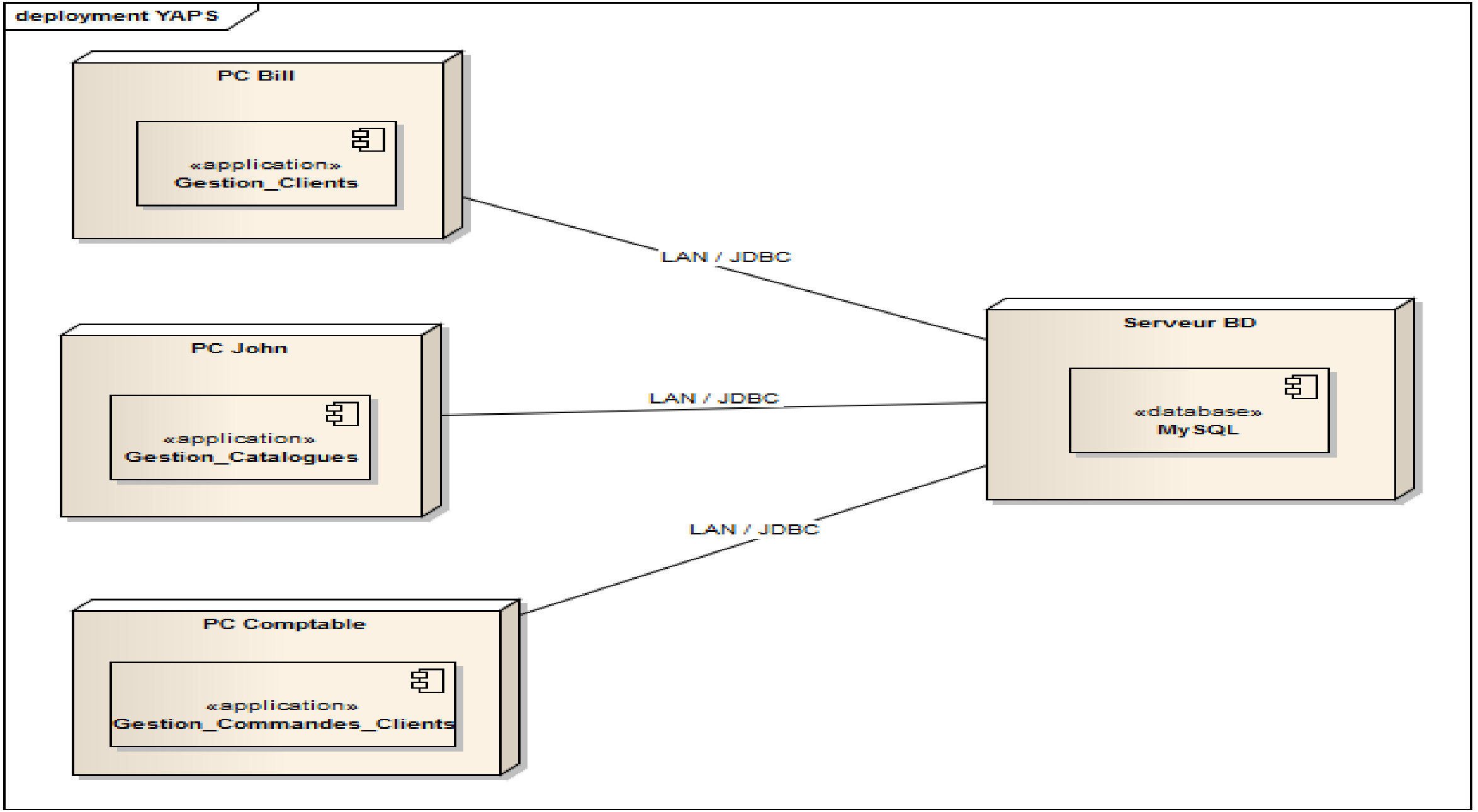
✓La société ne dispose pas d'un réseau local.


deployment YAPS
PC Bill
«application»
Gestion_Clients
PC Comptable
«application»
Gestion_Commandes_Clients
PC John
«application»
Gestion_Catalogues

➢ *Configuration 2:*

- YAPS décide d'investir dans un petit réseau local et d'avoir un poste distant accueillant la base de données. Ainsi, les deux applications gestion_Clients et Gestion_Catalogues utiliseront la même base de données distante.

Les besoins utilisateurs restent inchangés puisque cette nouvelle évolution est purement technique et non fonctionnelle. Les applications gestion_Clients et Gestion_Catalogues doivent continuer à fonctionner comme elles le faisaient auparavant.

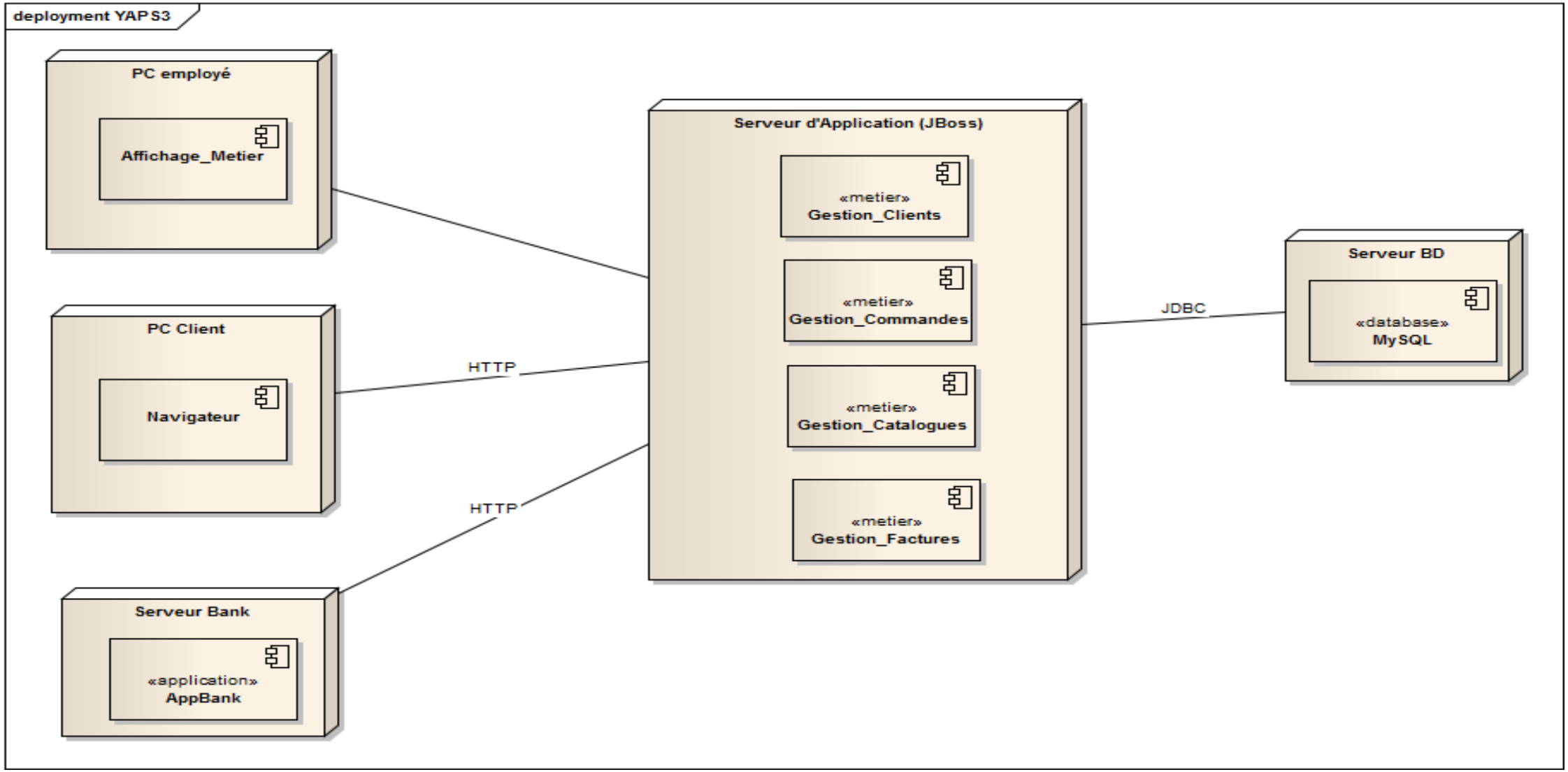

deployment YAPS
PC Bill
«application»
Gestion_Clients
LAN / JDBC
Serveur BD
«database»
MySQL
PC John
«application»
Gestion_Catalogues
LAN / JDBC
LAN / JDBC
PC Comptable
«application»
Gestion_Commandes_Clients

➢ *Configuration 3:*

- La société YAPS ne cesse de se développer. Elle a doublé ses effectifs et ses ventes ont augmenté de 200%. Les applications commencent à donner des signes de ralentissement. Les utilisateurs se plaignent de la lenteur des traitements.

Après avoir fait une étude sur son parc informatique, YAPS se rend compte que les postes utilisateurs sont sous-dimensionnés pour répondre à la demande croissante des clients.

- Moderniser tous les postes utilisateurs serait trop coûteux pour l'entreprise. YAPS décide alors d'investir dans un serveur puissant qui pourrait héberger la base de données et les traitements métiers de l'application.
- C'est donc une distribution de l'application qui est souhaitée. Les postes utilisateurs ne feraient plus aucun traitement mais simplement de l'affichage. Les traitements métiers et l'accès aux données seront effectués par le serveur à travers des appels distants. Cette évolution est purement technique. Elle doit permettre une meilleure utilisation des ressources informatiques des postes utilisateurs mais en aucun cas elle ne doit changer le comportement de l'application.
- Alors, pour résoudre ce problème le recourt à l’architecture 3-tiers MVC est bien clair, en séparant l’affichage, de traitement métier. Ces deux parties seront liées par les Contrôleurs qui font l’intermédiaire entre le métier et la partie Vue. Soit le diagramme de déploiement suivant alors :



On remarque que la partie métier « Gestion_Factures » est ajouté dans cette configuration, car ici on parle de système totalement informatisé. Donc l'encaissement des chèques sera réalisé à distance, c'est pour cela nous avons ajouté le serveur « Bank ».

Merci Pour Votre Attention